

How Computers Work · Section 12

Explore a Running System

By TOUCH DEN · November 11, 2026

Contents

What You Will Learn 1

Before You Start 1

1. The Goal: See the Ideas for Real 1

2. Who Am I, and What Is Running 2

3. Identifying and Reading Files 2

4. How Long Up, and How It Started 3

5. Tying It Together 4

Common Mistakes 4

Summary 5

Check Your Understanding 5

Practice 6

What You Will Learn

- How to see, on a real machine, the ideas from this whole course.
- How to inspect the running processes.
- How to identify files by content and read their raw bytes.
- How to see how long the system has been running since boot.
- How it all ties together into the way security people look at a computer.

Before You Start

Read all of lessons 1 to 11. This final lesson is a **project**: instead of new theory, you use what you have learned to explore a real, running computer with a few simple commands. The commands shown use Linux/macOS tools (`whoami`, `ps`, `file`, `xxd`, `uptime`); Windows has equivalents, noted along the way. Run these on **your own computer** only, the one rule from Security Fundamentals. Everything here is safe: you are *looking*, not changing. The outputs shown are from a real machine; yours will differ, which is exactly the point, you are seeing *your* system.

---

1. The Goal: See the Ideas for Real

You have built a complete mental model of a computer: bits and bytes, text encoding, the CPU, memory and storage, the operating system, processes, files, programs, boot, virtualization, and a first look at raw memory. This lesson makes it real. Everything you have learned is visible on the machine in front of you, with a handful of commands.

This is also how security people actually begin with an unfamiliar system. A defender triaging a possibly-compromised machine, or a forensic analyst examining one, asks the same basic questions you are about to ask:

- **Who am I, and what can I do here?** (users and permissions, lesson 5)
- **What is running?** (processes, lesson 6)
- **What are these files, really?** (files and magic bytes, lesson 7)
- **How long has this been up, and how did it start?** (boot, lesson 9)

By the end, you will have looked at a real system the way a professional does, and connected every observation to a concept from this course. Open a terminal on your own computer and follow along.

2. Who Am I, and What Is Running

Start with the most basic question, the one every attacker and defender asks first on a new system: **who am I, and what can I do?** The `whoami` command answers the first part:

```
$ whoami
user
```

That is your user account (lesson 5). On Linux/macOS, the related `id` command shows your groups, and whether you are the powerful **root** account; on Windows, `whoami` works too. This matters because, as lesson 5 taught, *what you are allowed to do depends on who you are*. A limited user and root see very different possibilities, which is why privilege escalation is an attacker's goal.

Next: **what is running?** From lesson 6, `ps` lists processes:

```
$ ps -o pid,ppid,%cpu,%mem,comm
  PID  PPID %CPU %MEM COMM
 1521     1   0.3  1.8 WindowServer
 2983  2970   1.2  4.1 Google Chrome
 3001  2983   0.0  2.2 Google Chrome Helper
 4102  3550   0.1  0.4 zsh
```

Read it with lesson 6's eyes: each process has a **PID**, a **parent** (**PPID**), and its CPU and memory use. Notice `Google Chrome Helper`'s parent is `Google Chrome`, the parent–child tree in action. On a typical machine there are hundreds of processes:

```
$ ps -e | wc -l
     900
```

Nine hundred processes, all sharing the CPU by the OS's multitasking (lesson 5), each isolated in its own virtual memory (lesson 6). (On Windows, open **Task Manager** to see the same thing.) This process list is where a defender hunts for the abnormal: a strange name, an unexpected parent, a process using too much CPU.

3. Identifying and Reading Files

Now turn to files on disk (lesson 7). First, identify what a file *really* is by its content, not its name, using `file` (which reads the magic bytes, lesson 7):

```
$ file greet
greet: Mach-O 64-bit executable arm64
```

`file` reports the true type by looking inside, here, a compiled executable (lesson 8), in this machine's Mach-O format (ELF on Linux, PE on Windows). It would say this even if the file were misleadingly named, which is exactly why defenders trust content over the extension.

Then read a file's raw bytes with `xxd` (lesson 2's hex dump):

```
$ printf 'Mina\n' | xxd
00000000: 4d69 6e61 0a                             Mina.
```

Every idea from lesson 2 and 3 is here: the **offset** on the left, the **bytes in hex** in the middle (`4d` = `M`, `69` = `i`, `6e` = `n`, `61` = `a`, `0a` = the newline), and the **text** on the right. You are reading memory the way a reverse engineer or forensic analyst does. And from lesson 8, remember that running `strings` on a program pulls the readable text straight out of it, a first step in analyzing anything unknown.

With just `file`, `xxd`, and `strings`, you can already look inside any file: what it is, what bytes it contains, and what readable clues it holds. That is the beginning of file analysis, forensics, and reverse engineering.

4. How Long Up, and How It Started

From lesson 9, the whole system came to life through the boot chain, firmware → bootloader → kernel → init → desktop, and everything running now descends from that boot. See how long ago it completed with `uptime`:

```
$ uptime
22:32  up 6 days, 23:47, 1 user, load averages: 1.31 1.43 1.70
```

"up 6 days" means the boot chain finished almost a week ago and the system has run continuously since. Every one of those hundreds of processes traces its ancestry back through **init** (PID 1, lesson 6 and 9) to that boot. The "1 user" is you (lesson 5); the load averages hint at how busy the CPU has been (lesson 4).

If you wanted to go deeper (and you will, in the Linux and defensive courses), the system keeps **logs** of what happened, including at boot and when programs start, which is where investigators reconstruct a timeline (lesson 7's timestamps, the forensics theme). You do not need those commands yet. The point here is the connection: a single `uptime` line ties back to the entire boot lesson, and reminds you that everything running now rests on a chain of trust from power-on.

5. Tying It Together

Step back and see what you just did. With five simple commands, you examined a real computer across the whole stack this course taught:

Command	What you saw	Lesson
`whoami` / `id`	Who you are; your privileges	5 (OS, users, permissions)
`ps`	The running processes, PIDs, parents, resource use	6 (processes)
`file`	A file's real type from its content	7, 8 (files, executables)
`xxd`	A file's raw bytes: offset, hex, text	2, 3 (binary, encoding)
`uptime`	How long since the boot chain completed	4, 9 (CPU, boot)

This is not a toy exercise; it is the real starting posture of security work. A **SOC analyst** or **incident responder** (courses 25, 27) begins triage with exactly these questions: who am I, what is running, what are these files, how and when did this system start. A **forensic analyst** (course 28) reads bytes and timestamps the same way. A **reverse engineer** (course 17) and **malware analyst** (course 29) use `file`, `xxd`, and `strings` as their first moves on an unknown program. You now share their basic vocabulary and their first instincts.

More importantly, you have proven to yourself that a computer is not magic. Every concept from this course, bits, encoding, the CPU, the OS, processes, memory, files, programs, boot, and virtualization, is real, visible, and inspectable with simple tools. That demystified, confident view of the machine is the true deliverable of this course, and the foundation everything ahead is built on.

Where you go next: the Linux course (03) turns these first commands into real command-line skill; the Networking course (04) adds how machines talk; and from there the paths open up, offensive, defensive, engineering. But whichever way you go, you will carry this lesson's habit: when you meet a new system, *look* at it, who, what, which files, how it started, and connect what you see to how computers actually work. You have finished How Computers Work. You understand the machine. Everything from here builds on that.

Common Mistakes

1. Running these on a machine that is not yours. Even "just looking" must be on your own system (or one you have permission for). The one rule never pauses.

2. Memorizing commands without the concepts. The value is connecting each command's output to how computers work. The commands are windows onto the ideas, not the point themselves.

3. Thinking a long process list is suspicious by itself. Hundreds of processes are normal. The skill is spotting the *abnormal* one, by name, parent, or resource use, against that normal background.

4. Trusting a possibly-compromised machine's own output. As lesson 9 warned, a rootkit can hide from these tools. On a truly suspect machine, professionals also examine it from outside.

5. Stopping here. This is a first look. The Linux and defensive courses turn these first commands into real investigative skill.

Summary

Idea	Remember
The project	Use simple commands to see this whole course on a real machine
`whoami` / `id`	Who you are and what you may do (users and permissions)
`ps`	What is running: PIDs, parents, resources (processes)
`file`	A file's real type, from content not name (files, executables)
`xxd` / `strings`	A file's raw bytes and readable text (binary, encoding, analysis)
`uptime`	How long since the boot chain completed (boot)
The real lesson	A computer is inspectable, not magic; this is how security work begins

Check Your Understanding

1. What four basic questions does a defender (and this project) ask about an unfamiliar system, and which lesson does each connect to?
2. What does `whoami` tell you, and why does it matter what account you are?
3. In a `ps` listing, what would make one process look suspicious among hundreds of normal ones?
4. Why do you trust `file`'s answer over a file's extension?
5. In the dump `4d69 6e61 0a Mina.`, which byte is the letter `n`, and what is `0a`?
6. What does "up 6 days" tell you, and what does every running process trace back to?
7. Why might a professional not fully trust the output of these commands on a truly compromised machine?

Answers

1. Who am I and what can I do (lesson 5, users/permissions); what is running (lesson 6, processes); what are these files really (lesson 7/8, files/executables); how long up and how did it start (lesson 4/9, CPU/boot).
2. Your user account. It matters because what you're allowed to do depends on who you are, a limited user and root have very different powers, which is why attackers try to escalate privilege.
3. A strange or unexpected name, an unusual parent (e.g. a document app whose child is a shell), or unusually high CPU/memory use, anything abnormal against the normal background.
4. Because the extension is just a changeable part of the name, while `file` reads the content's magic bytes, so it reports the true type even if the file is misleadingly named.
5. `6e` is the letter `n` (the third byte: `4d` M, `69` i, `6e` n, `61` a). `0a` is the newline character.
6. That the boot chain completed almost a week ago and the system has run continuously since. Every running process traces back through init (PID 1) to that boot.
7. Because a rootkit (lesson 9) runs with full privilege and can hide its own files and processes from the system's own tools, so the machine can lie about itself; serious work also examines it from outside.

Practice

This is the project. On **your own computer**, open a terminal (or Task Manager on Windows) and do the following, writing a short note connecting each to a lesson:

1. Run `whoami` (and `id` on Linux/macOS). Who are you? Are you an administrator/root or a limited user?
2. Run `ps -e | wc -l` (Linux/macOS) or open Task Manager (Windows). Roughly how many processes are running? Pick one and find its parent, does the relationship make sense?
3. Run `file` on two different files (e.g. a photo and a document), then `xxd` on a small text file. Does `file` match the extension? Can you read the offset, hex, and text columns in the dump?
4. Run `uptime` (Linux/macOS) or check uptime in Task Manager's Performance tab (Windows). How long since your machine booted?

Write one sentence per step saying which course concept it showed you. That write-up is a perfect first entry for the portfolio Security Fundamentals recommended.

A Worked Example

1. `whoami` shows my normal account, and `id` shows I'm not root, I'm a limited user, which (lesson 5) means malware running as me couldn't change the whole system.
2. `ps -e | wc -l` shows about 300 processes. I picked my terminal's shell and found its parent is the terminal app, which makes sense, the terminal launched the shell (lesson 6's process tree).
3. `file photo.png` says "PNG image data" and `file notes.txt` says "ASCII text", matching the content, not just the extension (lesson 7). `xxd notes.txt` shows the offset, the hex bytes, and the text, exactly lesson 2's three columns.
4. `uptime` says "up 2 days", so my machine completed its boot chain two days ago and every process traces back to it (lesson 9).

Course mark icon from Material Symbols by Google, Apache License 2.0.